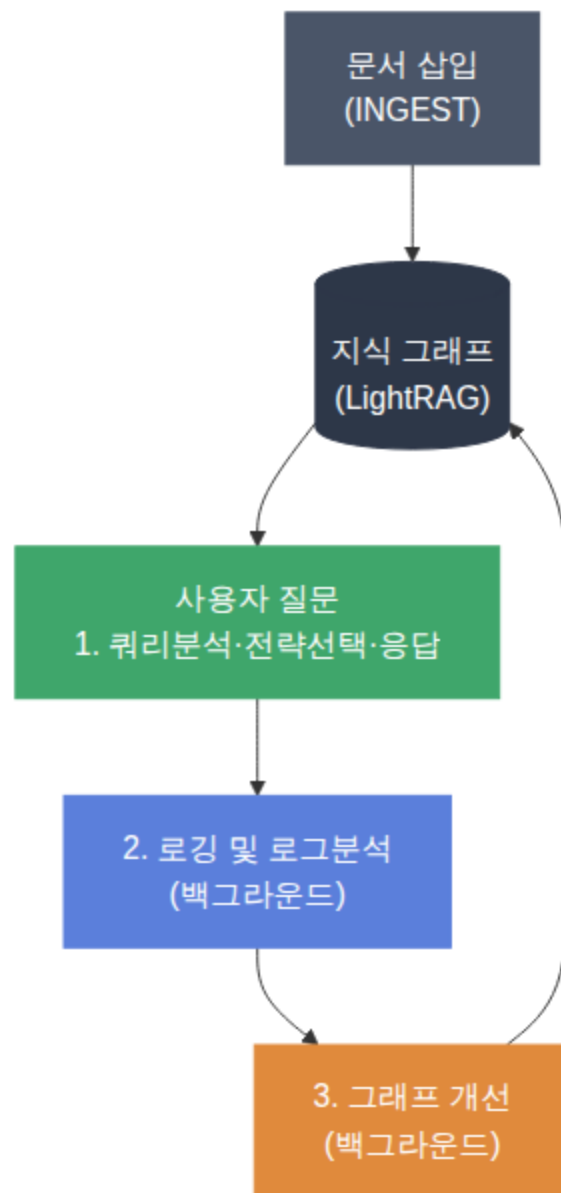


Evolving LightRAG

LightRAG 위에 쿼리 로그·리트리브 결과·코퍼스 메타데이터를 근거로 스스로 그래프를 넓혀가는 에이전트를 엮은 프로젝트입니다. Karpathy의 [LLM Wiki](#) 같은 "지식이 계속 정비되는 시스템"들에서 영감을 받았지만, 위키 페이지를 유지보수하는 게 아니라 LightRAG의 그래프 자체를 질문이 쌓일 때마다 계속 진화시키는 에이전트 RAG를 만든 것이라고 보면 됩니다.

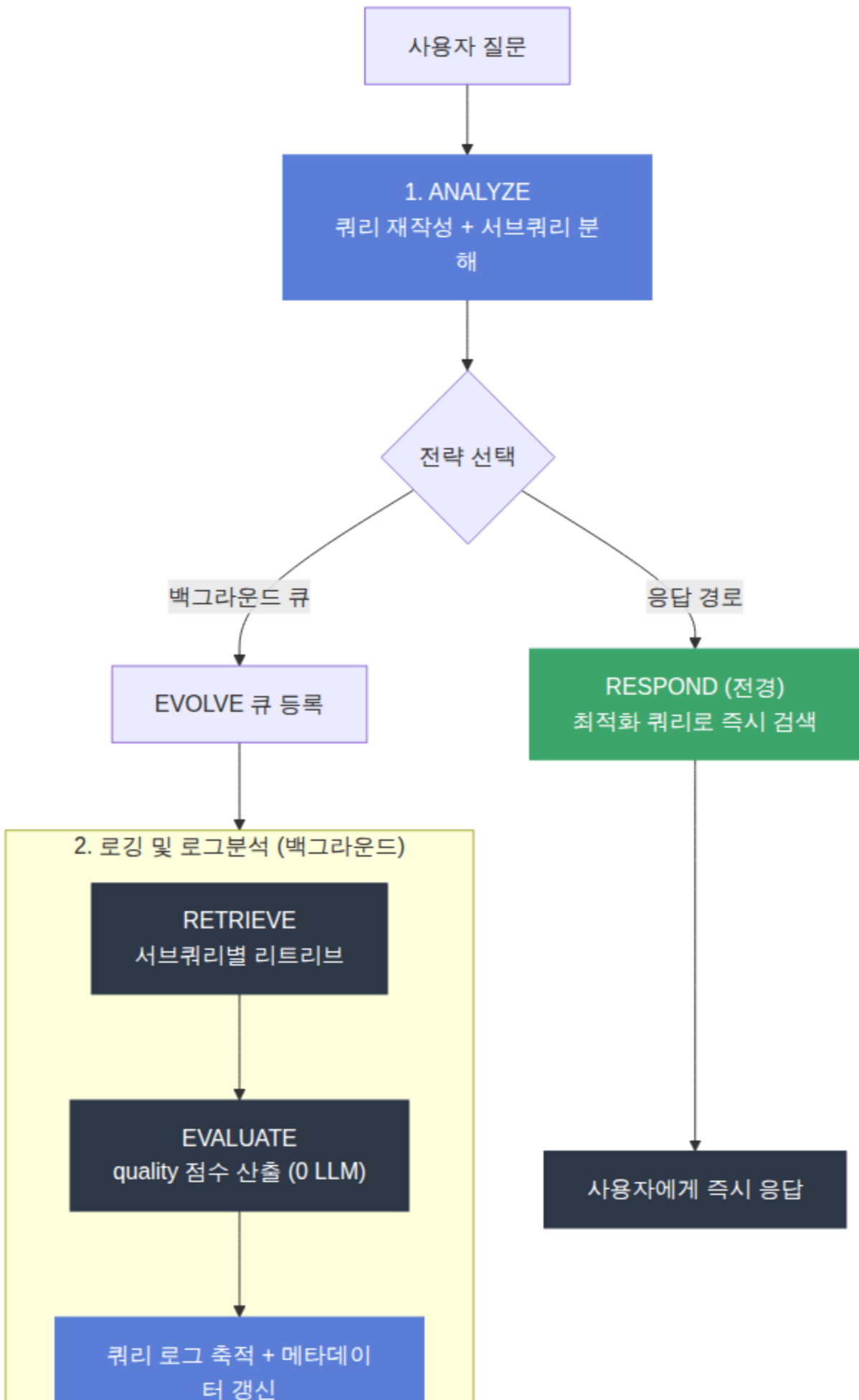
세부 전략·지표·이론적 배경은 이 문서에서 다루지 않습니다 — [ARCHITECTURE.md](#) 에 정리되어 있습니다.

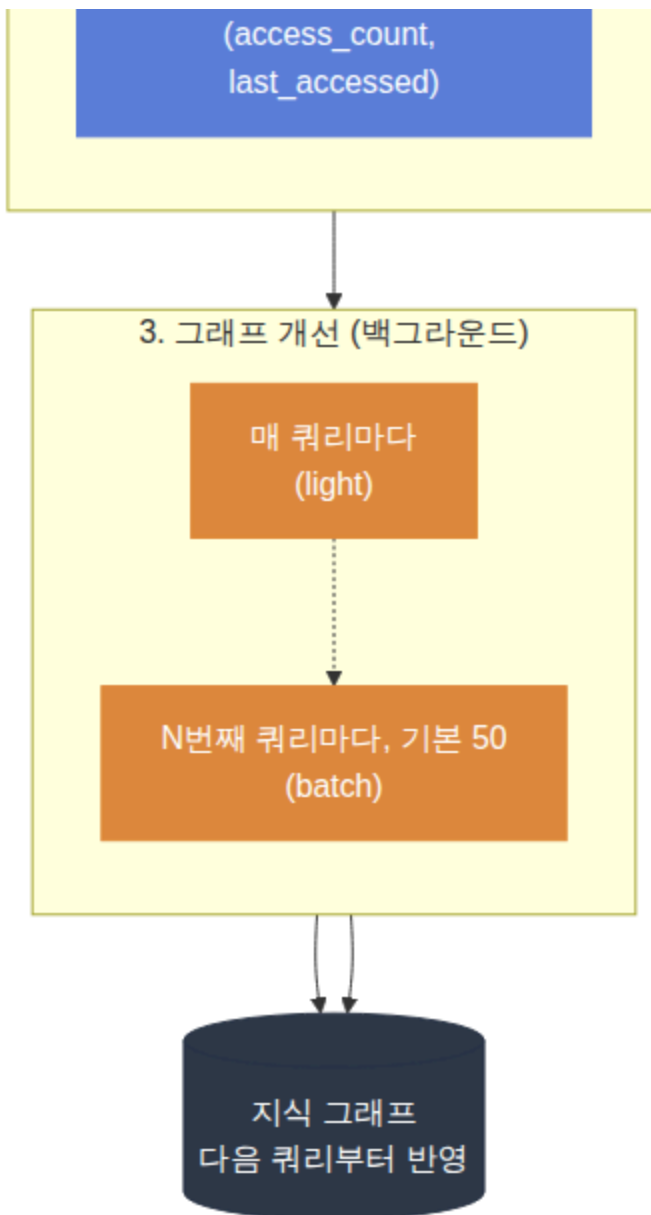


전체 흐름: 세 단계

에이전트가 하는 일은 크게 세 단계로 나뉩니다. 하나의 쿼리가 들어오면 1 → 2 → 3 순서로 이어지되, 2·3은 백그라운드에서 돌아 사용자 응답을 지연시키지 않습니다.

1. 쿼리분석 및 전략선택, 유저응답 — 질문을 검색에 최적화된 형태로 분석하고, 곧바로 사용자에게 답을 리턴합니다.
2. 로깅 및 로그분석 — 답변과 별개로 서브쿼리별 리트리브 결과를 채점하고, 쿼리 로그와 엔티티 메타데이터에 쌓습니다.
3. 그래프 개선 — 쌓인 로그와 코퍼스 메타데이터를 근거로 그래프에 엔티티/관계를 추가하거나 정리합니다.





1. 쿼리분석 및 전략선택, 유저응답

- ANALYZE: 질문을 검색에 최적화된 형태로 재작성하고, 하나의 질문이 여러 정보 요구를 담고 있으면 서브쿼리로 분해합니다.
- 전략 선택: ANALYZE의 출력을 곧바로 응답에 쓸지, 백그라운드 큐로 넘겨 로깅·그래프 개선까지 이어갈지 여기서 갈립니다. 실제로는 두 경로가 동시에 진행됩니다 — 응답을 막지 않습니다.
- RESPOND (유저응답): 최적화된 쿼리로 즉시 검색해 사용자에게 답을 리턴합니다. 로깅이나 그래프 개선을 기다리지 않습니다.

문서 스코핑: ANALYZE는 질문이 특정 문서(들)를 명시적으로 지칭하는지도 함께 판단합니다

(`mentioned_docs`). 3단계 그래프 개선(구조 마이닝)이 이미 그 문서에 대응하는 `doc::` 노드를 만들어 났다면, RESPOND는 그 문서가 `CONTAINS` 로 연결한 엔티티들로 검색 범위를 좁혀서 찾습니다 — 그래프 전체를 뒤지지 않고 지목된 문서 주변만 우선 탐색하는 것입니다. 문서 언급이 없는 평범한 질문이거나, 아

직 구조 마이닝이 그 문서를 못 봤다면(배치 주기 전이라 신선도가 안 맞을 수 있음) 그냥 평소처럼 전체 그래프를 검색합니다 — fail-safe.

구현은 [wikigraph/operations/analyze.py](#) (ANALYZE)와 [wikigraph/operations/query.py](#) (RESPOND, 문서 스코핑)를 참고하세요.

2. 로깅 및 로그분석

백그라운드 큐에 올라간 쿼리는 서브쿼리별로 다시 리트리브되고, 그 결과가 쿼리 로그와 엔티티 메타데이터로 쌓입니다. 이 단계에서 만들어진 로그가 다음 단계(그래프 개선)의 유일한 입력입니다.

- RETRIEVE: 서브쿼리마다 독립적으로 리트리브해서 "이 정보 요구에 대해 무엇이 검색됐는지"를 정확히 남깁니다.
- EVALUATE: 리트리브 결과마다 quality 점수(0.0~1.0)를 매깁니다. LLM 호출 없이 순수 규칙 기반입니다.
- 로그/메타데이터 축적: 쿼리 로그(`query_log` , 무엇이 자주 같이 검색됐는지)와 엔티티 메타데이터 (`access_count` , `last_accessed`)를 갱신합니다.

이렇게 쌓인 로그를 다시 훑어 패턴(co-retrieval 빈도, 반복되는 멀티홉 경로, 추출 누락)을 뽑아내는 것도 이 단계의 역할이며, 그 결과가 3단계 그래프 개선의 트리거·근거가 됩니다.

quality 점수란?

리트리브 결과가 얼마나 "쓸만했는지"를 나타내는 지표입니다. 그래프 개선 실행 여부를 막는 게이트는 아니고 — tier 1은 매 쿼리 백그라운드로 도니 항상 실행됩니다 — 로그 분석·모니터링용으로 남습니다.

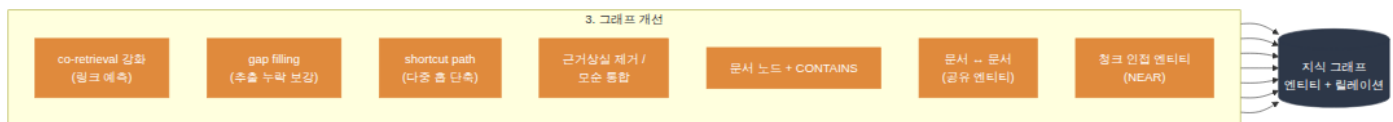
결과	quality
엔티티 0 + 청크 0	0.0
엔티티 있으나 관계 없음	0.3
청크만 있고 엔티티 없음	0.4
엔티티 + 관계 + 청크	0.5~1.0 (수량에 비례)

자세한 계산 규칙은 [ARCHITECTURE.md의 quality](#)를 참고하세요.

구현은 [wikigraph/operations/query.py](#) (RETRIEVE/EVALUATE)와 [wikigraph/metadata.py](#) (로그·메타데이터 영속화)를 참고하세요.

3. 그래프 개선

2단계에서 쌓인 로그·리트리브 결과, 그리고 LightRAG가 원래 갖고 있던 코퍼스 메타데이터 (`full_doc_id` , `chunk_order_index`)를 근거로 그래프를 확장(새 엔티티/관계 추가)하거나 정리(설명 통합, 근거 상실 관계 제거)합니다. 근거가 쿼리 로그 쪽이든 코퍼스 메타데이터 쪽이든 결국 하나의 카테고리입니다 — 모두 "그래프 개선"이라는 같은 백그라운드 파이프라인에서 함께 실행됩니다.



- co-retrieval 강화: 자주 함께 검색되는데 연결이 없는 엔티티를 링크 예측 방식으로 연결
- gap filling: 리트리브는 됐지만 추출이 누락된 청크에서 엔티티/관계를 재추출
- shortcut path: 반복되는 $A \rightarrow B \rightarrow C$ 다중 홉 경로를 $A \rightarrow C$ 로 단축
- 근거 상실 제거 / 모순 통합: 문서가 사라지면 근거를 잃은 관계를 지우고, 여러 출처의 모순된 설명을 하나로 통합
- 문서 노드 + CONTAINS: 문서를 그래프 노드로 만들고, 그 문서에서 추출된 엔티티를 연결
- 문서 ↔ 문서: 엔티티를 공유하는 문서끼리 연결
- 청크 인접 엔티티: 같은 문서의 인접한 청크에서 나온 엔티티끼리 연결

weight란?

원본 문서에서 추출된 관계와 그래프 개선이 추가한 관계를 구분하기 위해, 그래프 개선이 새로 만드는 모든 엣지에는 weight(엣지 가중치)가 낮게 매겨집니다. 검색 시 (rank, weight) 순으로 정렬되므로, weight가 낮을수록 원본 추출 관계보다 후순위로 밀립니다.

출처	weight
원본 문서에서 LLM 추출	1.0 (여러 문서에서 재추출되면 누적)
로그 기반 추론 (co-retrieval/gap-fill/shortcut)	0.5
코퍼스 메타데이터 기반 구조 마이닝	0.3

구조 마이닝 엡지가 더 낮은 이유: 항상 참이지만(문서가 실제로 그 엔티티를 포함), 의미적으로는 로그 기반 추론보다 느슨한 신호이기 때문입니다. 자세한 내용은 [ARCHITECTURE.md의 weight](#)를 참고하세요.

두 티어로 나뉩니다: 매 쿼리마다 가볍게 도는 tier 1(light)과, N번째 쿼리마다(기본 50) 누적된 로그 전체를 다시 훑어 더 폭넓게 개선하는 tier 2(batch)입니다. 그래프 전체를 스캔해야 하는 무거운 정리 작업(근거 상실 관계 제거, 모순 통합)과 문서/청크 구조 마이닝은 tier 2에 묶여서 함께 돕니다.

여기서 만들어진 변화는 이번 답변이 아니라 다음 쿼리부터 반영됩니다.

지식그래프 쪽 개선과 구조 마이닝은 코드도 하나의 모듈([wikigraph/operations/evolve.py](#))에 함께 있습니다 — 예전엔 `evolve.py` / `structural.py` 로 나뉘어 있었지만, 이제 문서 구조와 마찬가지로 "그래프 개선" 하나의 카테고리로 합쳤습니다.

전체 파이프라인은 [wikigraph/agent.py](#) 의 `WikiGraphAgent.query()` 가 담당합니다. `LangGraph StateGraph` 는 이 중 진짜로 여러 단계가 이어지는 백그라운드 파이프라인(`RETRIEVE` → `EVALUATE` → `EVOLVE(light)` → `EVOLVE(batch)` → `STRUCTURAL`)에만 쓰이고, `ANALYZE`와 `RESPOND`는 단일 호출이라 직접 함수로 호출합니다.

사용법

```
from wikigraph.agent import WikiGraphAgent
from wikigraph.config import WikiGraphConfig

rag = LightRAG(working_dir="./my_kb", llm_model_func=llm_func, embedding_func=embed_f
await rag.initialize_storages()

agent = WikiGraphAgent(rag, WikiGraphConfig(batch_evolve_interval=50), llm_func=llm_f

# 문서 삽입
await agent.ingest(["문서 내용..."])

# 질문: 즉시 답을 받고, 로깅·그래프 개선은 백그라운드 큐에 올라감
```

```

result = await agent.query("질문")
print(result["answer"])

# 백그라운드 작업이 끝나길 명시적으로 기다리고 싶을 때 (테스트/종료 시)
await agent.flush()

# 수동으로 그래프 개선 파이프라인 전체를 즉시 실행
await agent.evolve()

# 구조 점검 (린팅)
await agent.lint()

```

파일 구조

```

harness_pjt/
├─ wikigraph/
│   ├─ agent.py           # WikiGraphAgent: ANALYZE → fork(RESPOND / 백그라운드 로깅)
│   ├─ config.py          # 단계별 임계값
│   ├─ state.py           # LangGraph 상태 스키마 (TypedDict)
│   ├─ metadata.py        # 쿼리 로그 + 엔티티 메타데이터 영속화 (2. 로깅 및 로그분석)
│   ├─ sources.py         # 파일 수집, 변경 감지
│   └─ operations/
│       ├─ analyze.py     # 1. 쿼리 재작성 + 분해 + 문서 언급 감지(mentioned_docs)
│       ├─ ingest.py      # 문서 삽입 + 검증
│       ├─ query.py       # 1. RESPOND(전경, 문서 스코핑) + 2. RETRIEVE/EVALUATE(백
│       ├─ evolve.py      # 3. 그래프 개선 - 로그 기반(tier1/tier2) + 구조 마이닝, 현
│       └─ lint.py        # 린팅: 구조 점검 + 제한적 자동 정리
├─ diagrams/
│   └─ *.mmd              # 아래 다이어그램들의 mermaid 소스
├─ scripts/
│   └─ render_diagrams.py # diagrams/*.mmd → images/*.png (mermaid-cli)
└─ tests/
    └─ test_wikigraph.py

```

벤치마크 (수동 관찰치)

구성	정확도
vanilla LightRAG (그래프 개선 없음)	~69%
Evolving LightRAG (그래프 개선 적용)	64% ~ 72% (변동)

그래프 개선이 항상 vanilla보다 낫진 않습니다 — 구조 마이닝 엣지가 기존 지식그래프에 노이즈로 섞이는 것으로 보이며, 자세한 원인 분석과 대응 계획(구조 그래프 분리)은 [ARCHITECTURE.md의 벤치마크](#)를 참고하세요.

한계와 열린 질문

현재 구현은 아키텍처 제안과 프로토타입 단계입니다. 위 벤치마크에서 보듯 그래프 개선이 검색 품질을 항상 개선하는 건 아니고, 구조 관계가 멀티홉 질의에 실질적으로 도움이 되는지도 정량적으로 검증되지 않았습니다. 자세한 위험 분석과 TODO는 [ARCHITECTURE.md의 한계](#)를 참고하세요.

References

- [Karpathy's LLM Wiki](#) (2026.04)
- [LightRAG](#) (HKUDS, 2024)
- [On the Evolution of Knowledge Graphs: A Survey](#)

전략별 근거 논문은 [ARCHITECTURE.md의 References](#)에 정리되어 있습니다.